DECISIONS.md ΓÇö Architecture & Implementation Decisions

This document records architectural, design, and implementation decisions made during the development of the IT Resource & Operations Platform (IROP).

1. Authentication & Security

- **Decision**: Token-based authentication using JSON Web Tokens (JWT) signed with HMAC-SHA256 and passwords hashed using `bcryptjs` (salt rounds: 12).
- **Reason**: Stateless token verification allows clean horizontal API scaling, microservice readiness, and decoupled client-server architecture.
- **Client Storage**: In the initial client release, the JWT is stored in memory and persisted in `localStorage` with automatic Authorization headers injected via Axios interceptors. Future enterprise releases can transition to `httpOnly` secure cookies for additional defense in depth.
- **Account Registration**: No public signup. All accounts are created by administrators via `POST /api/users`. Inactive accounts (`isActive: false`) are blocked at login.

2. Role-Based Access Control (RBAC)

- **Decision**: Three distinct roles: `employee`, `manager`, `admin`.
- **Enforcement**: RBAC is enforced strictly at the API controller/middleware layer (`requireAuth` and `requireRole`). The client UI hides menus for convenience, but the API rejects unauthorized access with HTTP 403 Forbidden.
- **Hierarchy & Boundaries**:
  - `employee`: Access only their own leave balance, own leave history, and assets assigned to them.
  - `manager`: All employee privileges plus team leave approval queue (only direct reports where `managerId` equals manager ID) and read-only access to assets and office inventory.
  - `admin`: Full unrestricted access to user management, leave approvals across all departments, asset CRUD + assign/unassign, and inventory CRUD + +/- quantity adjustments.
- **Self-Approval Guard**: No user (even an Admin or Manager) can approve or reject their own leave requests.
- **Demotion/Deactivation Guard**: The last active admin in the system cannot be deactivated or demoted to prevent complete lockout.

3. Leave Management System (LMS)

- **Weekday Calculation**: Day counts between start and end dates inclusive strictly count Mondays through Fridays (skipping Saturdays and Sundays).
- **Deduction Timing**: Quota is only deducted when a leave request transitions from `pending` to `approved`. Rejections, cancellations, or applications do not deduct from the balance.

- **Atomic Deduction**: Quota deduction is performed via atomic MongoDB `$inc` update on `quotas.<type>.used` alongside updating leave status to prevent race conditions.
- **Date Constraints**: New requests cannot start in the past, with the exception of Work From Home (WFH) starting on the current calendar day.

---

4. Asset Management System (AMS)

- **Identifier**: `serialNumber` is unique and sparse (optional for un-serialized accessories, but unique when present).
- **Status Lifecycle**: Assigning an asset automatically updates status to `in_use`. Unassigning transitions status to `available`. Assets marked `in_repair` cannot be assigned until restored to `available`.
- **Search & Filter**: Server provides case-insensitive search across asset name and serial number with status filtering (`all`, `in_use`, `available`, `in_repair`).

---

5. Inventory Management System (IMS)

- **Threshold Rule**: `isLowStock` evaluates to `true` whenever `quantity <= threshold` (including when quantity equals threshold).
- **Floor Limit**: Item quantity cannot go below 0. Atomic `$inc` or validation guarantees positive bounds.
- **Role Permissions**: Admins have full CRUD and +/- 1 increment/decrement actions; Managers have read-only visibility; Employees do not have access to office inventory stock.

---

6. Database & Environment Configuration

- **Databases**: Development runs on MongoDB Atlas `irop_dev`. Automated test suite runs strictly on MongoDB Atlas `irop_test` (or local test database).
- **Zero Secrets Leakage**: Credentials and keys are restricted to `server/.env`. `.env.example` provides template placeholders.

---

7. Client & DevOps Architecture

- **Client Stack**: React 18, Vite, TypeScript, Tailwind CSS, TanStack Query, React Hook Form with Zod, Axios, Lucide React icons.
- **Container Strategy**: Multi-stage Docker builds on Node 20 Alpine and unprivileged Nginx, running under non-root users (`node` / `nginx`). Healthchecks verify container readiness via `/health`.